

Database Integration Pattern

A database tool server exposes a few narrow, safe tools rather than open-ended SQL execution:

Tool	Purpose
list_tables	Discover available tables
describe_table(table)	Inspect a table's columns and types
query(sql, params)	Run a read-only SELECT/WITH query, parameterized and row-limited

The AI is expected to:

1. Explore the schema first.
2. Then perform safe read-only queries.
3. Never perform writes.

The Security Model

Six guards are enforced and tested — each stops a specific attack:

1. Read-Only Database Connection
 - SQLite opened in mode=ro.
 - Any write operation fails automatically.
2. Allow-Listed Operations
 - Only SELECT and WITH statements are allowed.
3. Single-Statement Enforcement
 - Queries containing ; are rejected.
 - Prevents stacked-query attacks.
4. Parameterized Queries
 - Uses ? placeholders and bound parameters.
 - Prevents SQL injection.
5. Result Limits
 - Limits returned rows (MAX_ROWS).
 - Prevents excessive memory usage.
6. Identifier Validation
 - Table names are validated against actual database tables before being used.
 - Prevents injection through table or column names.

Hands-On Lab

Step 1 — seed_db.py (Create the Sample Database)

Create seed_db.py with the following code, then run it once to create store.db:

```
#!/usr/bin/env python3

"""Create a small SQLite database (store.db) for the Database Tool Server lab.

Run once before db_server.py: python seed_db.py
"""

import os
import sqlite3
from pathlib import Path

DB_PATH = Path(os.environ.get("DB_PATH", "./store.db")).resolve()

conn = sqlite3.connect(DB_PATH)
conn.executescript(
    """
    DROP TABLE IF EXISTS orders;
    DROP TABLE IF EXISTS products;

    CREATE TABLE products (
        id    INTEGER PRIMARY KEY,
        name  TEXT NOT NULL,
        category TEXT,
        price REAL
    );

    CREATE TABLE orders (
        id    INTEGER PRIMARY KEY,
        product_id INTEGER REFERENCES products(id),
        qty    INTEGER,
        customer TEXT
    );

    INSERT INTO products (name, category, price) VALUES
        ('Widget', 'hardware', 9.99),
        ('Gadget', 'hardware', 19.99),
        ('Manual', 'docs', 0.0);
```

```
INSERT INTO orders (product_id, qty, customer) VALUES
    (1, 3, 'alice'),
    (2, 1, 'bob'),
    (1, 5, 'carol');
"""
)
conn.commit()
conn.close()
print(f"Seeded database at {DB_PATH}")
```

Run the seed script:

```
python seed.py
```

Step 2 — server.py (The MCP Tool Server)

Create db_server.py with the complete server code below:

```
import json
import os
import sqlite3
import sys
from pathlib import Path

from mcp.server.mcpserver import MCPServer

DB_PATH = Path(os.environ.get("DB_PATH", "./store.db")).resolve()
MAX_ROWS = int(os.environ.get("MAX_ROWS", "100"))
ALLOWED_PREFIXES = ("SELECT", "WITH")

mcp = MCPServer("Database Tool Server")

def _connect() -> sqlite3.Connection:
    """Open a READ-ONLY connection. Any write attempt fails at the driver level."""
    if not DB_PATH.exists():
        raise ValueError(f"Database not found at {DB_PATH}. Run: python seed_db.py")
    conn = sqlite3.connect(f"file:{DB_PATH}?mode=ro", uri=True)
```

```
conn.row_factory = sqlite3.Row
return conn

def _ensure_safe(sql: str) -> str:
    """Allow only a single read-only SELECT/WITH statement."""
    stripped = sql.strip().rstrip(";").strip()
    if ";" in stripped:
        raise ValueError("Only a single statement is allowed (no ';').")
    if not stripped.upper().startswith(ALLOWED_PREFIXES):
        raise ValueError("Only read-only SELECT/WITH queries are allowed.")
    return stripped

@mcp.tool()
def list_tables() -> str:
    """List the user tables in the database."""
    conn = _connect()
    try:
        rows = conn.execute(
            "SELECT name FROM sqlite_master "
            "WHERE type='table' AND name NOT LIKE 'sqlite_%' ORDER BY name"
        ).fetchall()
    finally:
        conn.close()
    return json.dumps([r["name"] for r in rows])

@mcp.tool()
def describe_table(table: str) -> str:
    """Return the column definitions (name, type, not-null, primary-key) for a table."""
    conn = _connect()
    try:
        valid = {r["name"] for r in conn.execute(
            "SELECT name FROM sqlite_master WHERE type='table'"
        )}
        if table not in valid:
            raise ValueError(f"Unknown table: '{table}'")
        # 'table' is validated against real tables above; PRAGMA cannot be parameterized.
```

```

    cols = conn.execute(f"PRAGMA table_info({table})").fetchall()
finally:
    conn.close()
return json.dumps([
    {"name": c["name"], "type": c["type"], "notnull": c["notnull"], "pk": c["pk"]}
    for c in cols
])

@mcp.tool()
def query(sql: str, params: list[str] | None = None) -> str:
    """Run a read-only SELECT/WITH query.

    Use ? placeholders in `sql` and pass values in `params` to stay injection-safe,
    e.g. sql="SELECT * FROM products WHERE category = ?", params=["hardware"].
    At most MAX_ROWS rows are returned.
    """
    safe_sql = _ensure_safe(sql)
    args = params or []
    conn = _connect()
    try:
        cur = conn.execute(safe_sql, args)
        cols = [d[0] for d in cur.description] if cur.description else []
        rows = [dict(r) for r in cur.fetchmany(MAX_ROWS)]
    finally:
        conn.close()
    return json.dumps(
        {"columns": cols, "row_count": len(rows), "max_rows": MAX_ROWS, "rows": rows},
        default=str,
    )

if __name__ == "__main__":
    print(f"Database Tool Server (read-only) using {DB_PATH}", file=sys.stderr)
    mcp.run()

```

Test using client.py

```
import asyncio
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
import json

async def main():
    server_params = StdioServerParameters(
        command="python",
        args=["server.py"],
    )

    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:

            await session.initialize()

            print("\n=== Available Tools ===")
            tools = await session.list_tools()
            for tool in tools.tools:
                print(tool.name)

            print("\n=== 1. List Tables ===")
            result = await session.call_tool("list_tables", {})
            print(result.content[0].text)

            print("\n=== 2. SELECT Query ===")
            result = await session.call_tool(
                "query", {"sql": "SELECT name, price FROM products ORDER BY price DESC"}
            )
            json_result = json.loads(result.content[0].text)
            for row in json_result["rows"]:
                print(row)

            print("\n=== 3. Parameterized Query ===")
            result = await session.call_tool(
```

```
"query",
{
    "sql": "SELECT * FROM products WHERE category = ?",
    "params": ["hardware"]
}
)
json_result = json.loads(result.content[0].text)
for row in json_result["rows"]:
    print(row)

print("\n=== 4. Write Attempt (Should Fail) ===")
try:
    result = await session.call_tool(
        "query",
        {
            "sql": "INSERT INTO products(name) VALUES ('x')"
        }
    )
    print(result.content[0].text)
except Exception as e:
    print("Expected Error:", e)

print("\n=== 5. Stacked Query Injection (Should Fail) ===")
try:
    result = await session.call_tool(
        "query",
        {
            "sql": "SELECT 1; DROP TABLE products"
        }
    )
    print(result.content[0].text)
except Exception as e:
    print("Expected Error:", e)

print("\n=== 6. Table Name Injection (Should Fail) ===")
```

```

try:
    result = await session.call_tool(
        "describe_table",
        {
            "table": "products; DROP TABLE products"
        }
    )
    print(result.content[0].text)
except Exception as e:
    print("Expected Error:", e)

if __name__ == "__main__":
    asyncio.run(main())

```

Done When...

- seed_db.py created store.db; list_tables returns ["orders", "products"]
- A SELECT query returns rows; describe_table products returns columns
- INSERT / UPDATE / DELETE / DROP all return isError: true (allow-list)
- SELECT 1; DROP TABLE products returns isError: true (single-statement guard)
- An injection-style table name returns isError: true (identifier validation)
- Note all six security guards and which attack each one stops

We can Verify in the Inspector also:

Run each command below and confirm the expected output:

List tables

```

npx @modelcontextprotocol/inspector --cli mcp run db_server.py \
  --method tools/call --tool-name list_tables

```

Expected output:

```

"text": "[\"orders\\\", \"products\\\"]"

```

Run a SELECT query

```

npx @modelcontextprotocol/inspector --cli mcp run db_server.py \
  --method tools/call --tool-name query \
  --tool-arg "sql=SELECT name, price FROM products ORDER BY price DESC"

```

Expected output:


```
{"columns": ["name", "price"], "row_count": 3, "max_rows": 100,
"rows": [{"name": "Gadget", "price": 19.99}, {"name": "Widget", "price": 9.99},
{"name": "Manual", "price": 0.0}]}
```

Write blocked by allow-list

```
npx @modelcontextprotocol/inspector --cli mcp run db_server.py \
--method tools/call --tool-name query \
--tool-arg "sql=INSERT INTO products (name) VALUES ('x')"
```

Expected output:

```
"text": "Error executing tool query: Only read-only SELECT/WITH queries are allowed.",
"isError": true
```

Stacked-query injection blocked

```
npx @modelcontextprotocol/inspector --cli mcp run db_server.py \
--method tools/call --tool-name query \
--tool-arg "sql=SELECT 1; DROP TABLE products"
```

Expected output:

```
"text": "Error executing tool query: Only a single statement is allowed (no ';' ).",
"isError": true
```

Injection-style table name blocked

```
npx @modelcontextprotocol/inspector --cli mcp run db_server.py \
--method tools/call --tool-name describe_table \
--tool-arg "table=products; DROP TABLE products"
```

Expected output:

```
"text": "Error executing tool describe_table: Unknown table: 'products; DROP TABLE products'",
"isError": true
```

Parameterized query (injection-safe filtering)

Use ? placeholders with a params list. The value is bound by the driver, never concatenated into SQL:

```
npx @modelcontextprotocol/inspector --cli mcp run db_server.py \
--method tools/call --tool-name query \
--tool-arg "sql=SELECT * FROM products WHERE category = ?" \
--tool-arg "params=[\"hardware\"]"
```